



TÉCNICO
UNIVERSIDADE
DE LISBOA

Lightweight ADAS based on Vehicular Communication

PIC1 - Bachelor's in Telecommunications and
Informatics Engineering
Project Report

Diogo Filipe Torcato Folião
102970

Francisco Nuno Antunes Cardoso
109823

Department of Electrical and Computer Engineering
Instituto Superior Técnico

Date
28/06/2026

Abstract

This report presents a lightweight Advanced Driver Assistance System (ADAS) based on vehicle-to-vehicle (V2V) communication, developed for the 1st Cycle Integrated Project in Telecommunications and Informatics Engineering at Instituto Superior Técnico. While traditional driving aids rely on line-of-sight sensors like cameras or radar, our solution uses cooperative vehicular networks to add a crucial layer of safety redundancy and look beyond physical obstacles. The setup is built around two Accelerated Processing Units (APUs) configured as On-board Units (OBUs) running the ETSI C-ITS protocol stack via Vanetza. Telemetry data from external GPS modules is packaged into standardized Cooperative Awareness Messages (CAMs), which NAP-Vanetza automatically handles and translates into JSON format over MQTT. These packets are then forwarded via UDP to an offline Android dashboard application running on a Blackview Tab 60 tablet. By integrating the Mapsforge framework, the application renders localized OpenStreetMap vector data completely offline, displaying surrounding cars as dynamic, georeferenced polygons that match their actual dimensions and heading. To prevent accidents, the system runs predictive geometric analysis to project future vehicle paths, instantly triggering visual, acoustic, and haptic alerts for intersection collisions and blind-spot overtakings. Finally, the architecture was successfully validated through both simulated laboratory mockups and real-world road tests, proving that cooperative awareness can deliver low-latency hazard detection.

Table of Contents

Abstract	i
Table of Contents	ii
List of Abbreviations	iv
List of Figures	v
List of Tables	vi
1 Introduction	1
2 Technological Concepts and Frameworks	2
2.1 Vehicular Networks	2
2.2 Vanetza and MQTT Communication	3
2.3 Standardized Vehicular Communication Messages	4
2.3.1 Cooperative Awareness Message	4
2.4 Advanced Driver Assistance Systems	5
3 Solution	8
3.1 Hardware	8
3.2 System Architecture	8
3.3 Hardware Configuration	9
3.3.1 APU Interfaces	9
3.3.2 Network Interface Configuration	10
3.3.3 CAM Exchange Service	12
3.4 Android Environment and ADAS Application	14
3.4.1 Android Device Specifications	14
3.4.2 Android Development Environment	15
3.4.3 Mapsforge Integration and Offline Map Generation	15
3.4.4 Reception and Processing of CAM Messages	16
3.4.5 Vehicle Representation and Map Visualization	17
3.4.6 Driver Warning and Collision Detection System	18
4 Testing Methodologies	23
4.1 CAMs Test	23
4.2 Mocked Telemetry Data	24

4.3 On-the-field Tests	24
5 Conclusion	25
Bibliography	a
A Source Code	b
A.1 Subscriber Out Script (<code>subscriberOut.py</code>)	b
A.2 Subscriber Own Script (<code>subscriberOwn.py</code>)	c
A.3 Mock Collision Test Script (<code>mockCollisionTest.py</code>)	d
A.4 Collision detection	f
B AI Usage Declaration	h

List of Abbreviations

A DAS:	Advanced D river A ssistance S ystem
A PU:	Accelerated P rocessing U nits
C -ITS:	Cooperative Intelligent Transport Systems
C AM:	Cooperative A wareness M essage
D ENM:	Decentralized E nvironmental N otification M essage
E TSI:	European Telecommunications S tandards I nstitute
I TS:	Intelligent T ransport S ystems
ITS -S:	Intelligent Transport Systems Station
J SON:	J ava S cript O bject N otation
M APEM:	M ap D ata E xchange M essage
M QTT:	Message Q ueuing T elemetry T ransport
O BU:	O n-board U nit
P BF:	P rotocolbuffer B inary F ormat
R SU:	R oadside U nit
S PATEM:	S ignal P hase and T iming M essage
S SH:	S ecure S hell

List of Figures

Figure 2.1	Overview of the V2X communication ecosystem including vehicle-to-infrastructure (V2I) interactions.	2
Figure 2.2	MQTT publish-subscribe architecture mapping data exchange between an OBU and an RSU via a central broker	3
Figure 2.3	Structure of the CAM message.	5
Figure 2.4	Sensors used by the ADAS.	6
Figure 3.1	Diagram presenting the solution architecture.	9
Figure 3.2	Visualization of Diverse Idle Cars.	17
Figure 3.3	Collision warning interface displaying trajectory prediction lines and the visual alert banner.	19
Figure 4.1	Old Application interface displaying telemetry data from both stationary and moving APUs.	24

List of Tables

Table 3.1	Hardware components utilized in the vehicular communication system.	8
Table 3.2	Physical interfaces available on the APU board.	10
Table 3.3	Hardware characteristics of the target Android device.	14

1 | Introduction

This document consists of the project report developed within the scope of the 1st Cycle Integrated Project in Telecommunications and Informatics Engineering (PIC) course at Instituto Superior Técnico. This work comes as a direct response to the topic “Lightweight ADAS based on Vehicular Communication,” proposed by Professor Miguel Luís.

An ADAS (Advanced Driver Assistance Systems) consists of a set of technologies developed to improve road safety. Consequently, the proposed solution based on vehicular communication is of notable importance, as it develops tools that mitigate the physical limitations of drivers and serves as a complement to current driving support systems, which rely on gathering physical environment data through sensors. Thus, implementing a system that reinforces redundancy in detecting hazardous road scenarios aims to promote and increase the safety of all stakeholders.

The objective of this project is to implement a supporting system based on vehicular communication to improve the safety of all participants in road traffic. The project focuses on covering three key scenarios. Firstly, the solution provides accurate, real-time visual information about all vehicles in the surrounding area that support this communication protocol. Secondly, it predicts and warns the driver of potential collisions with other vehicles at intersections, which is highly valuable in low-visibility conditions. Lastly, it alerts the driver regarding blind-spot overtakings by other vehicles.

In this solution, an Android dashboard was developed, capable of representing multiple vehicles simultaneously and alerting the driver to different critical driving scenarios. To ensure data exchange between vehicles, communication was established using **A**ccelerated **P**rocessing **U**nits (APU). The technical infrastructure in use includes hardware configuration and message traffic management through specific software to route data packets to the mobile application.

The present report is structured to detail all development phases of the presented solution. In the first phase, the theoretical framework of the topic and the tools used are established. Next, the system architecture and technical implementation are described, including hardware configuration and the development of the Android dashboard. Finally, the test scenarios that validate the proposed solution are addressed, alongside the implementation challenges encountered. The report culminates with the project’s main conclusions.

2 | Technological Concepts and Frameworks

2.1 Vehicular Networks

A vehicular network is a specialized communication network where vehicles and roadside units act as nodes to exchange real-time data. They form the foundational technology of Intelligent Transportation Systems (ITS), enabling wireless communication between vehicles (V2V), road infrastructure (V2I or I2V), and pedestrians (V2P). This connectivity ecosystem is collectively known as V2X [1].

The communication relies on the installation of APUs on vehicles, specifically **O**n-board **U**nits (OBUs), which, in turn, communicate with **R**oadside **U**nits (RSUs) located on supporting infrastructure. This technology aims to improve road safety and traffic efficiency, as visually demonstrated in the following figure [2]:

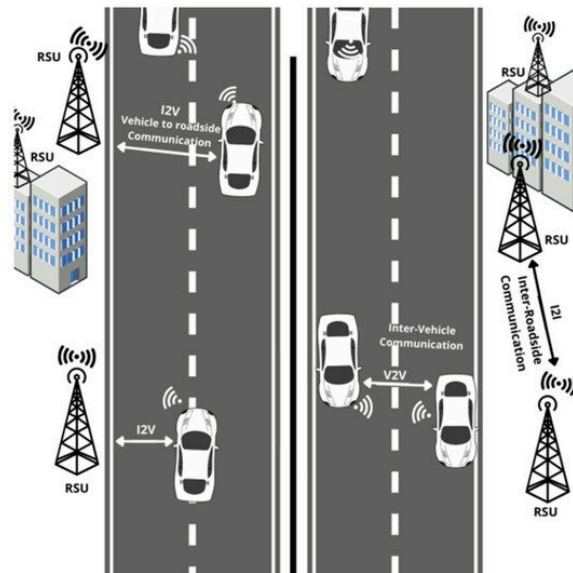


Figure 2.1: Overview of the V2X communication ecosystem including vehicle-to-infrastructure (V2I) interactions.

Currently, the implementation of these networks faces significant technical challenges, such as the difficulty of ensuring connectivity among all network participants due to their high mobility, or the need for sufficient processing capacity to process all data in real time while guaranteeing low latency. These networks must also consider the energy impact on

electric vehicle batteries and ensure rigorous security and privacy protocols to prevent data tampering or fake messages that could compromise driving and the safety of all vehicular network participants [1] .

Within the scope of this project, only two APUs provided by the faculty staff were used to simulate, study, and improve the road interaction between two vehicles. However, the current solution is implemented to support an arbitrary number of vehicles.

2.2 Vanetza and MQTT Communication

Vanetza [3] is an open-source implementation of the Cooperative Intelligent Transport Systems (C-ITS) protocol suite standardized by the European Telecommunications Standards Institute (ETSI). This communication stack encompasses fundamental protocols and functionalities for vehicular networks, notably GeoNetworking, Basic Transport Protocol, Decentralized Congestion Control mechanisms, and the underlying security layer. Although initially developed to operate over ITS-G5 channels, its modular architecture provides the flexibility required to seamlessly integrate with alternative communication technologies. To facilitate interoperability between the C-ITS stack and the high-level applications developed in this project, **NAP-Vanetza** is utilized [4].

This extension introduces native support for the **Message Queuing Telemetry Transport (MQTT)** [5] protocol along with **JavaScript Object Notation (JSON)** data formatting. MQTT is a lightweight, publish-subscribe messaging protocol designed for low-bandwidth, high-latency, or unreliable networks. Instead of establishing direct client-server connections, nodes communicate asynchronously through a central broker. **Sending Devices** distribute data by publishing messages to specific hierarchical paths called topics, while **receiving devices** subscribe to these topics to automatically catch relevant updates. This interaction using a RSU and an OBU can be seen in the following figure [4]:

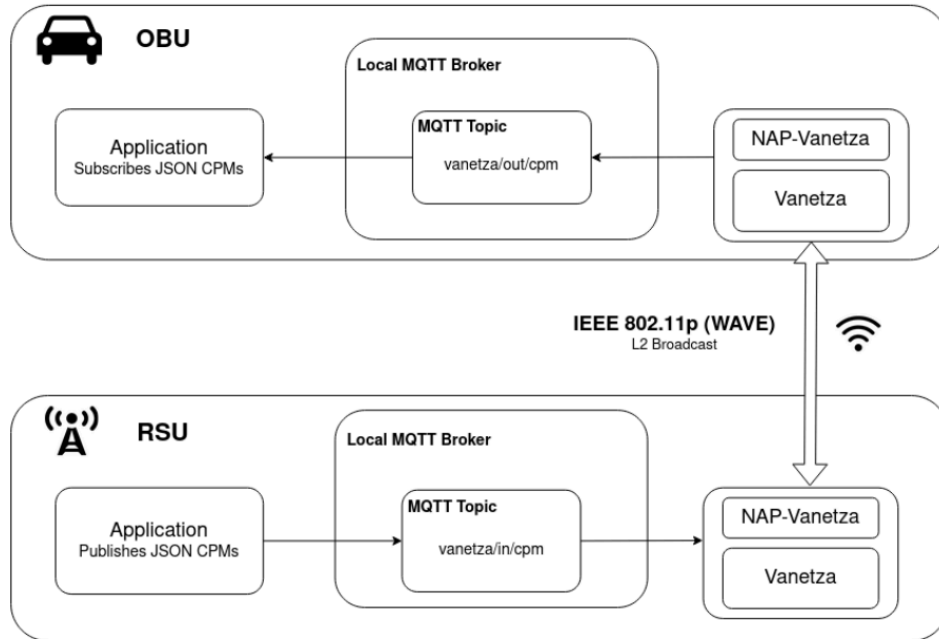


Figure 2.2: MQTT publish-subscribe architecture mapping data exchange between an OBU and an RSU via a central broker .

The primary objective of this extension is to abstract the inherent complexity of dealing with these network standards. NAP-Vanetza handles the encoding, decoding, transmission, and reception of various ETSI C-ITS message types, such as **Cooperative Awareness Messages (CAMs)**, relieving the developer from managing the low-level intricacies of the vehicular network.

With this architecture, the data flow becomes significantly more streamlined. For an application to transmit data externally, it only needs to construct a JSON representation of the message and publish it to the designated MQTT topic (in this project's case, `vanetza/own/cam`). Conversely, when neighboring vehicles broadcast messages, NAP-Vanetza captures the packets, decodes them, formats them into JSON, and publishes them to specific topics, such as `vanetza/out/cam`. The end application simply subscribes to these topics to gain structured and detailed access to the surrounding telemetry.

This approach successfully delegates the burden of message management to the NAP-Vanetza layer. Consequently, the application logic developed for this project could focus exclusively on processing, forwarding via UDP, and graphically representing the real-time data gathered from the vehicular network

2.3 Standardized Vehicular Communication Messages

The ETSI C-ITS architecture defines several standardized message types, each designed for a specific category of cooperative applications. Rather than relying on proprietary data formats, all **Intelligent Transport Systems Stations (ITS-Ss)** exchange standardized messages, ensuring interoperability between vehicles and equipment produced by different manufacturers.

Among the most widely used messages are:

- **CAM**, periodically broadcasting the current status of each vehicle;
- **Decentralized Environmental Notification Message (DENM)**, used to notify hazardous situations such as accidents, roadworks, or adverse weather conditions;
- **Signal Phase and Timing Message (SPATEM)**, providing traffic signal phase and timing information;
- **Map Data Exchange Message (MAPEM)**, describing road intersection topology and lane geometry.

Together, these messages enable cooperative awareness, event notification, infrastructure interaction, and advanced driver assistance functionalities.

2.3.1 Cooperative Awareness Message

Among the different ETSI **Intelligent Transport Systems (ITS)** messages, the **CAM** plays a central role by continuously providing information about the state and behavior of participating entities.

CAMs are periodically transmitted by ITS-Ss to establish and maintain mutual awareness within the vehicular network. They carry essential information regarding the originating ITS-S, including its current status and characteristics, with the exact message content depending on the type of station generating it.

A CAM contains a standardized collection of information describing both the transmitting station and its current dynamic state. Typical fields include:

- Station identifier;
- Latitude and longitude;
- Altitude;
- Vehicle heading;
- Speed;
- Longitudinal acceleration;
- Vehicle dimensions;
- Station type.

A more detailed structure of a CAM is presented in [Figure 2.3](#)

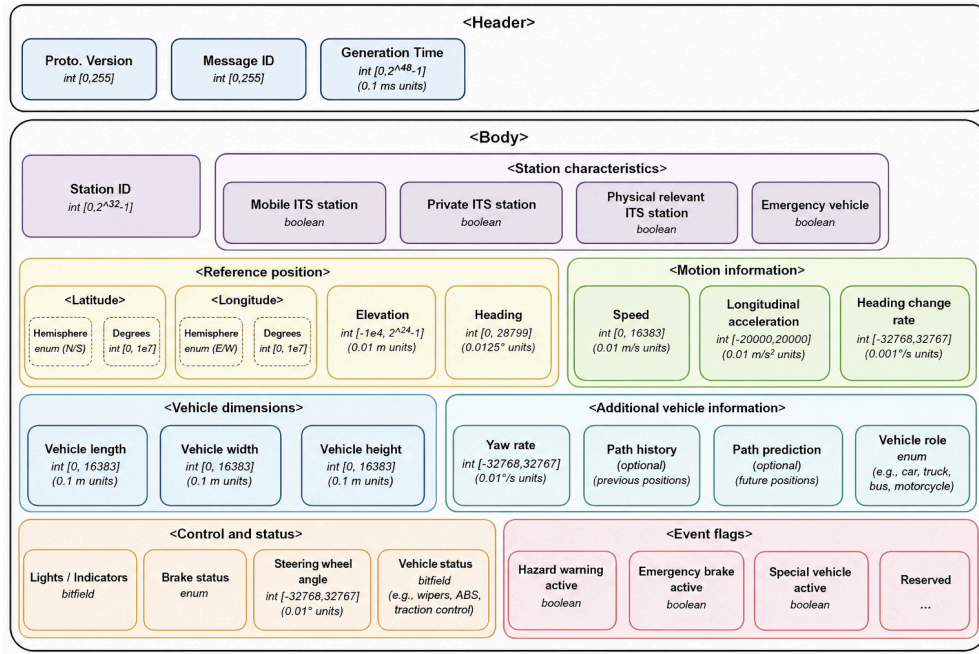


Figure 2.3: Structure of the CAM message.

These parameters allow neighboring vehicles to reconstruct an accurate representation of the surrounding traffic environment without relying exclusively on onboard sensors.

CAM generation follows a dynamic and event-driven approach, typically occurring at intervals ranging from 100 ms to 1000 ms. The transmission rate is adapted according to changes in the movement and dynamics of the originating ITS-Ss, ensuring that relevant and up-to-date information is shared with nearby participants.

2.4 Advanced Driver Assistance Systems

Modern **A**dvanced **D**river **A**ssistance **S**ystems (ADASs) combine information obtained from multiple sensing technologies to continuously estimate the state of the surrounding environment. Cameras are primarily used for lane detection, traffic sign recognition, and object classification. Radar sensors provide accurate distance and relative speed measurements,

particularly under adverse weather conditions. LiDAR sensors generate high-resolution three-dimensional representations of the environment, while ultrasonic sensors are commonly employed for low-speed manoeuvres such as parking assistance. Some of these mechanisms can be seen in Figure 2.4 reprinted from [6].

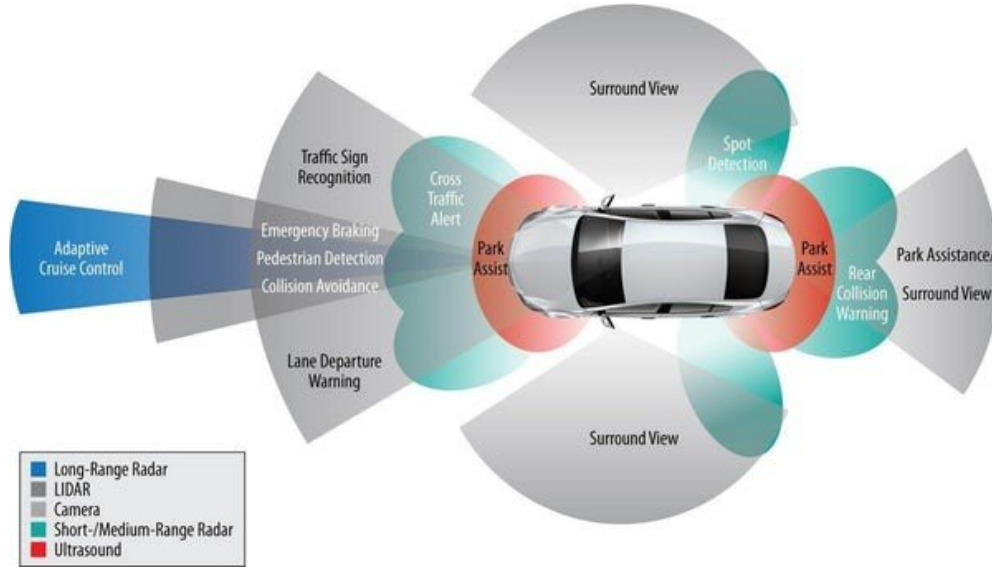


Figure 2.4: Sensors used by the ADAS.

The information collected by these sensors is processed by computer vision and sensor fusion algorithms, enabling the vehicle to identify surrounding vehicles, pedestrians, cyclists, lane markings, and potential obstacles.

Depending on the vehicle manufacturer and equipment level, ADAS may include:

- Automatic Emergency Braking;
- Adaptive Cruise Control;
- Lane Keeping Assist;
- Blind Spot Detection;
- Rear Cross Traffic Alert;
- Traffic Sign Recognition;
- Driver Drowsiness Detection;
- Parking Assistance;
- Collision Warning Systems.

Many of these systems operate continuously and intervene only when a potentially dangerous situation is detected, either by alerting the driver or by automatically controlling the steering or braking systems.

Although highly effective, perception-based ADAS remain limited by the sensing range and visibility conditions of their onboard sensors. Curves, large vehicles, buildings, adverse weather, and other obstacles may prevent the vehicle from detecting hazards located outside its direct line of sight.

Cooperative vehicular communication addresses this limitation by allowing vehicles to exchange information beyond the sensing capabilities of onboard sensors. Rather than relying exclusively on local perception, vehicles become capable of receiving information

from neighboring ITS stations that may already have detected hazards outside their own field of view. This extended situational awareness represents one of the main motivations behind C-ITSs and constitutes the foundation upon which the solution developed in this project is built.

3 | Solution

3.1 Hardware

To provide a clear overview of the physical setup developed for this project, the essential hardware components utilized in the deployment of the vehicular communication system are summarized below. [Table 3.1](#) outlines each component, its respective quantity, and its specific purpose within the dual-vehicle prototyping environment.

Component	Quantity	Purpose
APU Board	2	Two units (one per vehicle) running the Vanetza C-ITS software stack to manage low-level vehicular networking.
Wireless Antenna	2	Externally connected to the APU boards to enable IEEE 802.11p (WAVE) L2 Broadcast communication.
GPS Module	2	Externally connected to the APU boards to obtain and share precise vehicle telemetry and positioning data.
Android Tablet	1	Connected to one of the APUs to serve as the user interface, displaying real-time visual information and alerts.
Ethernet Cable	1	Establishes the physical network connection between the APU board and the tablet.
Ethernet-to-USB-C Adapter	1	Adapts the Ethernet cable connection to the USB-C interface of the tablet.
Powerbank	2	Power the units to allow on-the-field usage.

Table 3.1: Hardware components utilized in the vehicular communication system.

3.2 System Architecture

The presented solution consists of two OBU (APU) boards, with one placed in each vehicle, communicating with each other via wireless communications using the IEEE 802.11p (WAVE) protocol in Layer 2 Broadcast. Each APU runs the Vanetza software stack to manage low-level vehicular network complexities.

Communication between the boards is carried out via two externally added antennas, and they share the information obtained through a GPS module that is also externally connected

to the respective boards. This telemetry data is streamlined through NAP-Vanetza, which handles the encoding, decoding, transmission, and reception of CAM messages. The application interacts with this system by subscribing and publishing JSON-formatted data to specific MQTT topics, such as `vanetza/own/cam` for outbound transmissions and `vanetza/out/cam` for incoming telemetry from neighboring vehicles.

The diagram representing the system's architecture is presented in [Figure 3.1](#). The Android tablet running the application is directly connected to APU 144, while APU 101 is used to represent an external car.

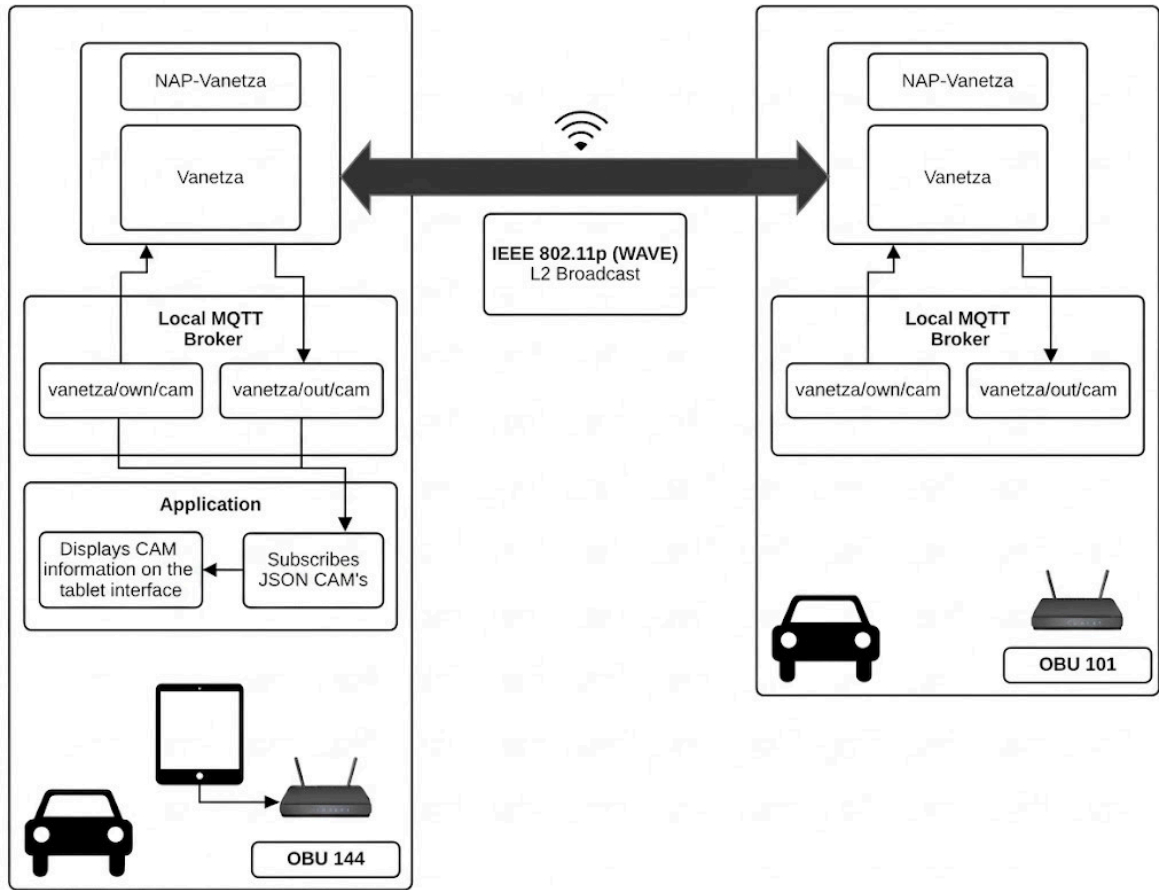


Figure 3.1: Diagram presenting the solution architecture.

3.3 Hardware Configuration

3.3.1 APU Interfaces

To detail the connectivity of the APU, the available physical interfaces on the board are described below. [Table 3.2](#) itemizes each port type, its available quantity, and its specific role in the project deployment.

Interface / Port	Quantity	Purpose
Ethernet (RJ-45)	3	Used for local network configurations, including the connection to the Android tablet via adapter.
USB	2	Available for general-purpose peripherals and external hardware expansion.
VGA	1	Provides video output for debugging and local system monitoring if required.
Antenna Port (Male)	1	Connects to the external wireless antenna to enable the IEEE 802.11p (WAVE) communication.
Antenna Port (Female)	1	Secondary antenna connector used to complement the RF interface and signal reception.
Power Entry	1	Physical connector to plug in an external power cable

Table 3.2: Physical interfaces available on the APU board.

Although the APU board provides a diverse set of interfaces, only a specific subset was utilized to implement the system. Specifically, the project strictly required the use of the male and female antenna ports to establish the wireless communication link, along with two of the available Ethernet ports to handle the wired network connections for the implementation of the solution and configuration purposes.

3.3.2 Network Interface Configuration

To access and configure the APU boards via network, it is necessary to understand their physical port mapping and virtual static IP addressing scheme. The APU features three physical Ethernet ports, each configured with virtual interfaces that follow a deterministic mathematical pattern based on the unique identifier of the board (APU_{ID}) [7].

The network interfaces are mapped as follows:

- **ETH0 (Left Port):** Dynamically obtains an IP address via DHCP. Additionally, it hosts the virtual interface **ETH0.1** with a 23 subnet mask. The IP address structure 192.168.AA.BB is derived using:

$$AA = \left\lfloor \frac{\text{APU}_{\text{ID}}}{255} \right\rfloor + 4 \quad (3.1)$$

$$BB = \text{APU}_{\text{ID}} \bmod 255 \quad (3.2)$$

Example: For board **NAP-157**, $AA = \left\lfloor \frac{157}{255} \right\rfloor + 4 = 4$ and $BB = 157 \bmod 255 = 157$, resulting in the static IP 192.168.4.157.

- **ETH1 (Middle Port):** Hosts the virtual interface **ETH1.1** with a 23 subnet mask. The IP address structure 192.168.CC.DD is derived using:

$$CC = \left\lfloor \frac{\text{APU}_{\text{ID}}}{255} \right\rfloor + 6 \quad (3.3)$$

$$DD = \text{APU}_{\text{ID}} \bmod 255 \quad (3.4)$$

- **ETH2 (Right Port):** Hosts the virtual interface **ETH2.1** with a 23 subnet mask. The IP address structure 192.168.EE.FF is derived using:

$$EE = \left\lfloor \frac{\text{APU}_{\text{ID}}}{255} \right\rfloor + 8 \quad (3.5)$$

$$FF = \text{APU}_{\text{ID}} \bmod 255 \quad (3.6)$$

3.3.2.1 APU Interface Configuration

In order to configure the APU, it was necessary to access it via **Secure Shell** (SSH). Consequently, the configuration process was initiated on APU 144 using one of its Ethernet interfaces.

To accomplish this, an Ethernet cable was connected directly to the ETH0 interface, using an Ethernet-to-USB-C adapter to link it to the USB-C port of an external computer. Based on the IP address computations detailed in this Section, the ETH0 interface of this specific APU is assigned the static IP 192.168.4.144. Because no DHCP server was running on this port and the computer needed to reside within the same subnet to establish communication, the IP address 192.168.4.1 was manually assigned to the computer's USB-C interface.

After configuring the computer's interface to match the APU's subnet, the following command was executed in the terminal to gain access to the APU operating system: `ssh nap@192.168.4.144`

3.3.2.2 Tablet Interface Configuration and Access

However, connecting an Android tablet to the system introduced distinct network constraints. Due to the inherent limitations of the Android environment regarding manual network management, combined with the requirement for a plug-and-play solution every time the system is powered on, manually assigning a static IP address upon every boot was neither practical nor feasible. To overcome this limitation and automate the network configuration for the tablet, it became necessary to deploy a **DHCP server**, ISC DHCP Server [8], on the APU interface. For this purpose, the Ethernet 1 (ETH1) interface was selected.

Since only a single Ethernet cable was available, it was not possible to maintain the connection to the configuration computer while simultaneously connecting the APU to a wired Internet source. Therefore, to install the DHCP server, it was necessary to share the computer's Wi-Fi connection with its Ethernet port, enabling the APU to access the Internet through the bridged interface.

Once Internet access was established on the APU, the following command was executed to install the required software package: `apt-get install isc-dhcp-server`

Subsequently, the main server configuration file was edited using: `sudo nano /etc/default/isc-dhcp-server` to specify the physical interface that would host the DHCP service: `INTERFACESv4="eth1"`.

Next, the subnet parameters were defined by modifying the `/etc/dhcp/dhcpd.conf` configuration file as follows:

```
1  subnet 192.168.6.0 netmask 255.255.254.0 {
2      range 192.168.6.10 192.168.6.50;
3      option routers 192.168.6.144;
4      option domain-name-servers 8.8.8.8, 8.8.4.4;
5      default-lease-time 600;
6      max-lease-time 7200;
7  }
```

After restarting the server and checking its status, it was possible to verify that the service was operating correctly.

3.3.3 CAM Exchange Service

As previously explained in Section 2.2, the data retrieved by the GPS module is wirelessly exchanged between the APUs in a JSON format using the `vanetza/out/cam` and `vanetza/own/cam` subtopics. Therefore, it was necessary to design a mechanism to collect all of this telemetry information stored in APU 144 and forward it to the Android application.

Two Python [9] scripts were developed to enable the communication of these messages between the Android tablet and the APU: **subscriberOut.py** and **subscriberOwn.py**. The former is responsible for managing the flow of external CAM messages received from other vehicles, whereas the latter aims to provide information regarding its own vehicle, in this case, the vehicle hosting APU 144.

As described in Section 3.3.2, a DHCP server was configured in the Ethernet1 port. It is important to note that the assigned ip addresses given by the server will be in the range from 192.168.6.10/23 to 192.168.6.50/23. This way, since the Android tablet will be the only device requesting for an IP at the given port, it is correct to assume that its IP address will be 192.168.6.10. Moreover, the port 5000 was chosen to send the UDP packages contained the telemetry data.

The `subscriberOut.py` script, omitting the debugging messages, is detailed below:

```
1  import paho.mqtt.client as mqtt
2  import json
3  import datetime
4  import socket
5
6  BROKER = "localhost" # local MQTT
7  TOPIC = "vanetza/out/cam" # Topic to be subscribed
8  IP_DESTINO = "192.168.6.10" # Destination IP
9  PORTA_DESTINO = 5000
10
```

```

11 sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM) # UDP Socket creation
12
13 def on_connect(client, userdata, flags, rc):
14     if rc == 0:
15         print(f"[{datetime.datetime.now()}] Connected to Broker!")
16         client.subscribe(TOPIC)
17         print(f"Subscribed to: {TOPIC}")
18     else:
19         print(f"Connection error: {rc}")
20
21 def on_message(client, userdata, msg):
22     try:
23         payload_str = msg.payload.decode()
24
25         # Sends message via UDP
26         sock.sendto(msg.payload, (IP_DESTINO, PORTA_DESTINO))
27
28     except Exception as e:
29         print(f"Error processing/sending message: {e}")
30
31 # Client Initialization
32 client = mqtt.Client()
33 client.on_connect = on_connect
34 client.on_message = on_message
35
36 print(f"Connecting to {BROKER}...")
37 client.connect(BROKER, 1883, 60)
38 client.loop_forever() # Keeps the script running

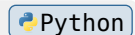
```

While multiple vehicles might transmit information every 100 ms to the APU running the application, it is critical for the update rate to be instantaneous. That is, each car's information must be forwarded to the application the exact moment it arrives at the `vanetza/out/cam` topic. Conversely, only a single vehicle publishes data to the `vanetza/own/cam` topic. Because there is no need to continuously stream redundant data into the application, a 1 second delay was introduced. The `subscriberOwn.py` script is structured similarly to the previous one, with the exception of the subscribed topic, the variable `last_update_time` and the `on_connect` callback function, which contains the following logic:

```

1  last_update_time = 0 # Time control
2
3  def on_message(client, userdata, msg):
4      global last_update_time
5      current_time = time.time()
6
7      # 1 second interval
8      if current_time - last_update_time < 1.0:
9          return # Ignores the messages until 1 second has passed
10

```



```

11     last_update_time = current_time
12     try:
13         payload_str = msg.payload.decode()
14
15         # Sends message via UDP
16         sock.sendto(msg.payload, (IP_DESTINO, PORTA_DESTINO))
17
18     except Exception as e:
19         print(f"Error processing/sending message: {e}")

```

The whole script is presented in [page b](#).

After verifying the correct transmission of telemetry data into the APU, a systemd service bundling these two files was created and configured to be deployed every time the APU is powered on.

3.4 Android Environment and ADAS Application

3.4.1 Android Device Specifications

The Android application developed within the scope of this project was deployed and evaluated on a Blackview Tab 60 tablet. This device was selected as the target platform due to its portability, adequate computational capabilities, and compatibility with the networking and visualization requirements of the system.

The tablet runs DokeOS_P 4.2, a customized Android-based operating system built upon Android 15. The device is equipped with a Unisoc Tiger T310 processor featuring four CPU cores operating at frequencies of up to 2 GHz. It also provides 4 GB of LPDDR4 physical memory, with support for memory expansion through virtual RAM mechanisms.

Regarding visualization capabilities, the tablet features an 8.68-inch IPS display with a resolution of 800×1340 pixels and a 16:10 aspect ratio. The display supports a refresh rate of 60 Hz and provides sufficient brightness for operation under different environmental conditions. These characteristics make the device suitable for the real-time visualization of geographic information and vehicular awareness data.

The hardware specifications of the selected platform are summarized in [Table 3.3](#).

Component	Specification
Operating System	DokeOS_P 4.2 (Android 15)
Processor	Unisoc Tiger T310, Quad-Core, up to 2 GHz
RAM	4 GB LPDDR4
Display	8.68" IPS
Resolution	800×1340 pixels
Refresh Rate	60 Hz

Table 3.3: Hardware characteristics of the target Android device.

3.4.2 Android Development Environment

The Android application developed in this project was implemented using Android Studio, the official Integrated Development Environment for Android development. Android Studio provides a complete development ecosystem, including the Android SDK, debugging and profiling tools, an integrated emulator for testing virtual devices, and native support for the Gradle build system.

The project was developed using Kotlin, the programming language officially recommended by Google for Android development. Kotlin provides modern language features, improved code readability, and full interoperability with existing Java libraries, making it particularly suitable for the development of reliable and maintainable Android applications.

The application targets a minimum Android SDK level of API 24, corresponding to Android 7.0 (Nougat). This version was selected to ensure compatibility with a wide range of Android devices while still providing access to the APIs and functionalities required by the project.

In addition to the Android development environment itself, offline geographic data preparation was necessary to provide local map resources.

3.4.3 Mapsforge Integration and Offline Map Generation

The visualization component of the application relies on **Mapsforge**, an open-source mapping framework specifically designed for offline map rendering on Android devices. Unlike solutions that depend on online map services, Mapsforge allows geographic information to be stored locally and rendered directly on the device, eliminating the need for Internet connectivity during operation.

Mapsforge uses a proprietary binary map format (**.map**) optimized for efficient storage and rendering. This format contains vector-based geographic information extracted from OpenStreetMap datasets and is particularly suitable for embedded and mobile systems where storage capacity and network connectivity may be limited.

To generate the map resources required by the application, OpenStreetMap data for Portugal was obtained from the Geofabrik distribution service [10], which provides regularly updated extracts of OpenStreetMap data in **Protocolbuffer Binary Format (PBF)**. The downloaded dataset contained the complete geographic information for Portugal and served as the basis for the offline map generation process.

Due to the size of the original dataset and the memory requirements associated with the conversion process, the geographical area of interest was first extracted using the Osmium tool [11]. This preprocessing stage reduced the volume of data to be processed while retaining all relevant information required by the application.

The extraction process was performed using a geographic bounding box covering the intended operational area:

```
1 osmium extract \  
2 -b -9.8,36.8,-6.0,42.2 \  
3 portugal-260323.osm.pbf \  
4 -o portugal-limited.osm.pbf
```



After the extraction stage, the resulting PBF file was converted into the Mapsforge binary format using Osmosis [12] together with the Mapsforge Map Writer plugin. The conversion process generated a `.map` file optimized for local storage and efficient rendering on Android devices.

```
1 osmosis \
2 --read-pbf file=portugal-limited.osm.pbf \
3 --mapfile-writer file=portugal.map
```



The generated map file is packaged with the application and accessed locally during execution. This approach eliminates the dependency on external map providers and ensures that geographic information remains available even in environments where Internet connectivity is unavailable or unreliable.

The adoption of Mapsforge therefore contributes to the robustness of the proposed system by enabling fully offline map visualization while maintaining low storage requirements and efficient rendering performance on resource-constrained Android devices.

3.4.4 Reception and Processing of CAM Messages

The Android application receives vehicular information through a UDP socket listening on port 5000. This communication channel acts as the interface between the vehicular communication infrastructure and the visualization layer implemented on the tablet.

A dedicated background thread continuously listens for incoming UDP datagrams in order to avoid blocking the user interface. Each received packet contains a JSON representation of a CAM, previously decoded and forwarded by the communication subsystem.

Upon reception, the application parses the JSON message and extracts the relevant information required for visualization and hazard detection, namely:

- Station identifier (`stationID`);
- Geographic position (`latitude` and `longitude`);
- Vehicle speed;
- Vehicle heading;
- Vehicle length;
- Vehicle width.

The heading value follows the ETSI C-ITS representation, where the heading is transmitted as an integer with a resolution of 0.1 degrees. Consequently, the received value is converted to degrees before being used by the visualization algorithms. Vehicle dimensions are also extracted directly from the CAM message whenever available.

The station identifier is used to distinguish the host vehicle from neighboring vehicles. Messages associated with the local vehicle update the current map position and camera focus, while messages originating from neighboring vehicles update or create new visual entities on the map.

To maintain an up-to-date representation of the surrounding environment, each neighboring vehicle is associated with a timestamp corresponding to the last received message. Vehicles that do not transmit updates for more than three seconds are automatically removed from the visualization layer, preventing stale information from remaining visible on the map.

3.4.5 Vehicle Representation and Map Visualization

Vehicle visualization is implemented using the Mapsforge overlay system. Rather than representing vehicles as simple markers, the application renders each vehicle as a georeferenced polygon whose dimensions and orientation correspond to the information transmitted in the CAM messages.

For every received vehicle position, a rectangular polygon is generated using the vehicle center coordinates, **length**, **width**, and **heading**. The polygon vertices are computed in geographic coordinates by converting metric offsets into latitude and longitude displacements and applying a rotation according to the vehicle heading.

This approach provides a significantly more realistic representation of the vehicle footprint and orientation than conventional point-based markers. As a result, the driver can immediately identify not only the location of neighboring vehicles but also their direction of travel and approximate physical dimensions.

Different colors are used to distinguish the local vehicle from neighboring vehicles. The host vehicle is represented by a blue polygon, while surrounding vehicles are displayed using red polygons.

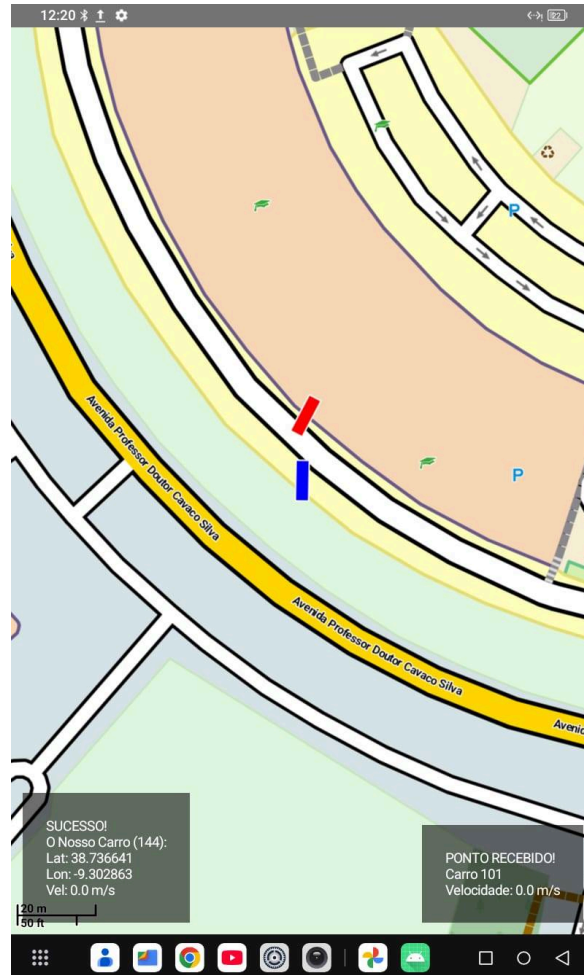


Figure 3.2: Visualization of Diverse Idle Cars.

In addition to the vehicle polygons, the application generates auxiliary graphical elements intended to support situational awareness. A prediction line is projected in front of each vehicle based on its current speed and heading, providing a short-term estimation of its

future trajectory. The local vehicle also includes a rear transversal reference line that is used by the overtaking detection algorithm.

These graphical elements form the basis for the Advanced Driver Assistance System functions implemented in the application. Collision warnings are generated whenever trajectory prediction lines intersect neighboring vehicle polygons or predicted trajectories. Overtaking warnings are generated when a neighboring vehicle intersects the rear reference line while travelling in approximately the same direction.

Whenever a hazardous situation is detected, the application activates visual, audio, and haptic feedback mechanisms. These include warning messages displayed on screen, colored border overlays, vibration patterns, and audible alerts, ensuring that the driver is immediately informed of potentially dangerous situations.

3.4.6 Driver Warning and Collision Detection System

One of the primary objectives of the developed application is to provide the driver with early warnings regarding potentially hazardous situations detected through cooperative vehicular communication. To achieve this, the application continuously analyses the position, dimensions, heading, and speed of all vehicles currently known through the received CAMs.

When a hazardous situation is detected, the application presents a warning through two complementary mechanisms:

- A prominent red warning banner displayed at the top of the application interface, as seen in [Figure 3.3](#);
- An audible alert signal reproduced through the Android device.

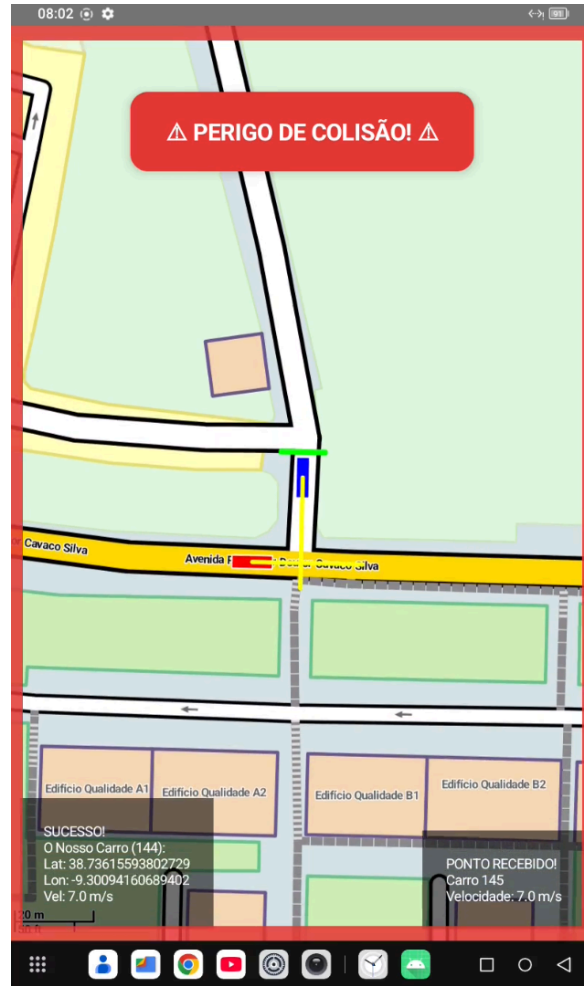


Figure 3.3: Collision warning interface displaying trajectory prediction lines and the visual alert banner.

The simultaneous use of visual and acoustic notifications increases the likelihood that the driver becomes aware of the warning, even when not directly observing the screen.

The warning remains active while the collision condition persists and is automatically removed once the vehicles are no longer considered to be on a collision course.

The warning system operates entirely on the Android device and is based on predictive geometric analysis. Rather than considering only the current positions of vehicles, the application estimates their near-future trajectories and evaluates whether those trajectories may result in dangerous interactions.

3.4.6.1 Future Trajectory Projection

For each vehicle represented on the map, the application computes a projected trajectory based on the information contained in the received CAMs. The projection considers:

- Current geographical position;
- Vehicle heading;
- Instantaneous speed;
- Vehicle dimensions.

$$P(t) = P_0 + v * t * \hat{d} \quad (3.7)$$

where P_0 is the current vehicle position, v is the instantaneous speed, t is the prediction time, and $\hat{d}$ is the unit vector representing the vehicle heading.

The resulting trajectory is represented graphically by a yellow line extending from the front of the vehicle in the direction of travel, as seen in [Figure 3.3](#). This line indicates the estimated position that the vehicle will reach after a predefined prediction interval, which was configured to four seconds in the current implementation.

$$t_p = 4s \quad (3.8)$$

The start and end coordinates of the resulting segment are then used by the collision detection algorithm to perform geometric intersection calculations with the trajectories and polygons of neighboring vehicles

$$S = (P_0, P(t_p)) \quad (3.9)$$

where S represents the prediction segment extending from the current position to the estimated future position

Initially, the trajectory visualization was introduced as a debugging tool to facilitate the verification of the collision detection algorithms. However, it also provides a clear visual representation of the prediction model used internally by the application.

3.4.6.2 Collision Detection

The collision detection mechanism continuously evaluates the geometric relationship between the projected trajectories of all vehicles present in the system. Rather than analyzing the visual representation displayed on the screen, the application performs the detection using the geographic coordinates of the trajectory segments computed for each vehicle.

For every vehicle, a *prediction segment* is generated from the current vehicle position to the estimated future position obtained from the trajectory projection algorithm. The application then applies line-segment intersection calculations to determine whether the projected path of the local vehicle intersects the projected path of another vehicle. This test is implemented in the function `checkCollisions`, which is provided in [page f](#).

$$\text{orient}(a, b, c) = (c_y - a_y)(b_x - a_x) - (c_x - a_x)(b_y - a_y) \quad (3.10)$$

AB intersects CD if:

$$\text{orient}(A, C, D) * \text{orient}(B, C, D) < 0 \wedge \quad (3.11)$$

$$\text{orient}(A, B, C) * \text{orient}(A, B, D) < 0 \quad (3.12)$$

A warning condition is generated whenever one of the following situations is detected:

- The prediction segment of the local vehicle intersects the prediction segment of another vehicle;

- The prediction segment of the local vehicle intersects the physical area occupied by another vehicle.

$$\text{Collision} = I(S_{\text{ego}}, S_{\text{others}}) \vee I(S_{\text{ego}}, P_{\text{others}}) \quad (3.13)$$

where S_{ego} is the prediction segment of the local vehicle, S_{others} represents the prediction segment of a neighboring vehicle, and P_{others} denotes the polygon describing the occupied area of that vehicle

To accurately represent the occupied space, each vehicle is modelled as a polygon whose dimensions are obtained directly from the corresponding CAM message. This allows the collision detection process to consider the actual size of vehicles rather than treating them as dimensionless points.

$$P_{\text{vehicle}} = \{p_1, p_2, \dots, p_n\} \quad (3.14)$$

where $p_1, p_2, \dots, p_n$ are the vertices defining the vehicle polygon.

$$I(S, P) = \text{exists } i : S \text{ intersects } (p_i, p_{i+1}) \quad (3.15)$$

Using the vehicle polygons significantly improves the realism of the detection process. Potential collisions can therefore be identified even when the predicted trajectories do not intersect directly but would still result in a physical overlap between vehicles.

Whenever a collision risk is identified, the application immediately generates a visual and audible warning to attract the driver's attention.

3.4.6.3 Overtaking Detection

In addition to frontal collision detection, the application also implements a mechanism for identifying potential overtaking situations.

For this purpose, a second geometric reference is associated with the local vehicle. This reference is represented on the map by a green line positioned perpendicular to the rear section of the vehicle

$$L = (P_r, P_r + n) \quad (3.16)$$

where P_r is the reference point located at the rear of the local vehicle and n is the unit normal vector perpendicular to the vehicle heading.

The green line defines a virtual detection boundary behind the vehicle. The application evaluates whether neighboring vehicles cross this boundary while travelling in approximately the same direction.

$$\Delta\theta = |\theta_{\text{ego}} - \theta_{\text{other}}| \quad (3.17)$$

It is considered 'same direction' if:

$$\min(\Delta\theta, 360^\circ - \Delta\theta) < 30^\circ \quad (3.18)$$

By combining the relative position, heading, and movement of neighboring vehicles, the system can identify situations where another vehicle is rapidly approaching from behind and entering a potential overtaking path

As with the trajectory visualization, the green line is displayed on the map primarily as a diagnostic and validation aid, allowing the behavior of the detection algorithm to be observed in real time during testing.

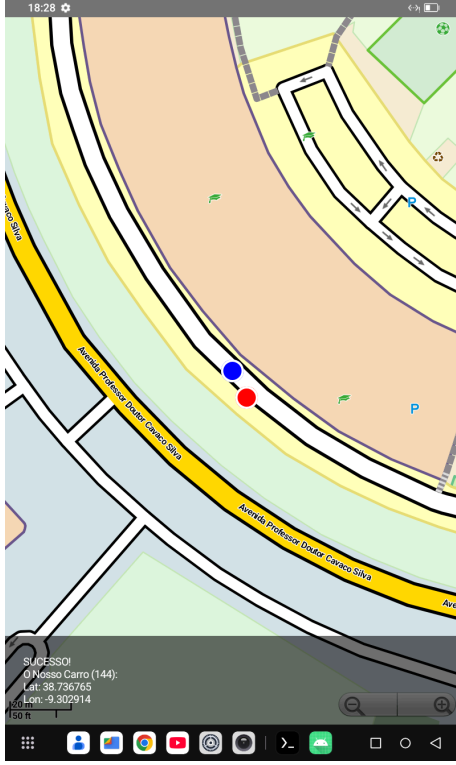
4 | Testing Methodologies

In order to test the implemented solution, three distinct approaches were selected. First, the reception and effectiveness of CAMs were evaluated on the running Android application alongside some basic on-the-field tests. Second, vehicle movements were simulated and tested in the laboratory using mocked telemetry data. Finally, full-scale on-the-field tests involving vehicles were conducted in the presence of the supervising professor.

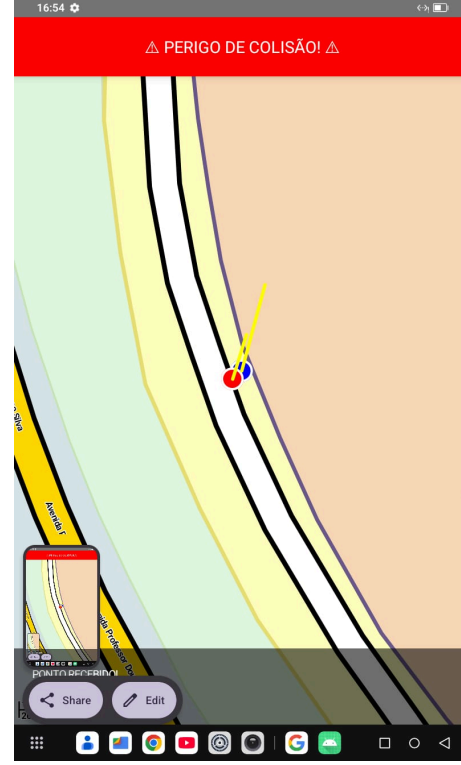
4.1 CAMs Test

To test if the CAMs were arriving at the application correctly, it was first necessary to verify whether they were reaching the designated port (5000). To achieve this, **Termux** [13] was installed on the Android device, which is an application that enables the use of a Linux terminal environment within an Android system. Following a successful installation, the **ncat** [14] utility, a versatile command-line tool used for reading, writing, and redirecting data across a network, was utilized. This software allowed the monitoring and reading of all formatted JSON messages sent via UDP from the APU to the tablet's designated port.

After verifying that the packets were arriving correctly, on-the-field tests were conducted on the first version of the presented solution. In order to verify the correct rendering of APU 144 and APU 101 on the screen and before using any kind of vehicle, tests on foot were carried out across the facilities of the Oeiras Campus of Instituto Superior Técnico. The correct display of the telemetry information related to the stationary boards is shown in [Figure 4.1a](#), while the figure on its right demonstrates the correct display when the boards are moving at a certain velocity.



(a) Stationary APUs.



(b) Moving APUs.

Figure 4.1: Old Application interface displaying telemetry data from both stationary and moving APUs.

4.2 Mocked Telemetry Data

Before conducting the on-road tests, a mockup interaction between vehicles was simulated in the laboratory. By dynamically modifying the CAM coordinates, velocity, and heading angle, it was possible to simulate two vehicles approaching each other at an intersection close to the campus to validate the collision avoidance mechanism. This test is implemented in the script `mockCollisionTest.py`, which is provided in [page d](#). This interaction runs in a loop, and the behavior can be observed in the following link:

<https://youtube.com/shorts/Vm2z26nHtUg?si=mR-WCGAHuC7Jx8qq>.

Additionally, the overtaking scenario was also simulated with mock data, which can be seen in the following link: https://youtube.com/shorts/BQni_sQkMiA. To confirm that the overtaking warning is only triggered when the vehicles share the same heading, the collision detection tool was deactivated and an additional simulated test was conducted with two cars passing in close proximity in opposite directions. The results of this test can be observed at the following link: <https://youtube.com/shorts/LL5qZSebqHU>.

4.3 On-the-field Tests

To validate the solution, the final test was conducted with vehicles on the road. Both the collision avoidance and the blind-spot overtaking scenarios were tested and successfully validated in the presence of the supervising professor. The recording of the conducted scenario can be accessed via the following link:

5 | Conclusion

This project successfully developed a lightweight ADAS that uses V2V communication to keep drivers safer on the road. By building this system around APU boards and its communication, we showed that cooperative networks can easily step in where traditional sensors like cameras or radar fall short, more specifically when looking through physical obstacles or blind corners.

On the technical side, the implementation proved that the used tools can be highly practical. Using the Vanetza framework, we managed to abstract low-level network complexities into simple JSON packets. By setting up an automated local DHCP server on the APU and writing Python scripts, we created a seamless, real-time data pipeline straight to our Android application.

Our predictive algorithms to compute the intersection collision detection and the blind-spot overtaking alerts performed exactly as intended. We thoroughly tested the system through laboratory simulations using mocked data, and then proved its real-world capability during on-the-field vehicle tests with the professor, where it delivered instant visual, audio, and haptic warnings without noticeable lag.

While deploying V2X networks on a massive scale still comes with complex real-world hurdles like high-speed mobility and heavy data processing, this project delivers a solid, functional, and reproducible open-source prototype. The final architecture fulfills all our initial goals, demonstrating that cooperative vehicle communication is an incredibly powerful tool for boosting driver awareness and adding a critical layer of safety redundancy.

Bibliography

- [1] Instituto Superior Técnico, “MScThesisJoaoViegas2024.pdf.” 2024.
- [2] M. Sindhwani *et al.*, “Soft Computing Techniques Aware Clustering-Based Routing Protocols in Vehicular Ad Hoc Networks: A Review,” *Applied Sciences*, vol. 12, no. 15, p. 7922, 2022, doi: [10.3390/app12157922](https://doi.org/10.3390/app12157922).
- [3] Vanetza Project Contributors, “Vanetza: An Open-source Implementation of the ETSI ITS-G5 Protocol Suite.” [Online]. Available: <https://github.com/riehl/vanetza>
- [4] Instituto Superior Técnico, “Vanetza - Introduction and Containerized Usage Guide __ Wiki IT NAP-Av.pdf.”
- [5] OASIS Standard, “MQTT Version 5.0. OASIS Standard.” [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [6] B. Strbac, M. Gostovic, Z. Lukac, and D. Samardzija, “YOLO Multi-Camera Object Detection and Distance Estimation,” May 2020, pp. 26–30. doi: [10.1109/ZINC50678.2020.9161805](https://doi.org/10.1109/ZINC50678.2020.9161805).
- [7] Instituto Superior Técnico, “[Mobility] APU Network Interfaces Info __ Wiki IT NAP-Av.pdf.”
- [8] Internet Systems Consortium, “ISC DHCP Server Documentation.” 2026. [Online]. Available: <https://www.isc.org/dhcp/>
- [9] Python Software Foundation, “Python Language Reference, Version 3.” 2026. [Online]. Available: <https://www.python.org/>
- [10] OpenStreetMap, “Geofabrik.” 2026. [Online]. Available: <https://download.geofabrik.de/europe/portugal.html/>
- [11] Jochen Topf, “Osmium Command Line Tool.” 2026. [Online]. Available: <https://osmcode.org/osmium-tool/>
- [12] OpenStreetMap, “Osmium Command Line Tool.” 2026. [Online]. Available: <https://wiki.openstreetmap.org/wiki/Osmosis/>
- [13] Termux Project, “Termux Wiki and Documentation.” 2026. [Online]. Available: <https://termux.dev/>
- [14] Nmap Project, “Ncat Users' Guide.” 2026. [Online]. Available: <https://nmap.org/ncat/>

Appendix A | Source Code

A.1 Subscriber Out Script (subscriberOut.py)

```
1  import paho.mqtt.client as mqtt
2  import json
3  import datetime
4  import socket
5
6  BROKER = "localhost" # local MQTT
7  TOPIC = "vanetza/out/cam" # Topic to be subscribed
8  IP_DESTINO = "192.168.6.10" # Destination IP
9  PORTA_DESTINO = 5000
10
11 sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM) # UDP Socket creation
12
13 def on_connect(client, userdata, flags, rc):
14     if rc == 0:
15         print(f"[{datetime.datetime.now()}] Connected to Broker!")
16         client.subscribe(TOPIC)
17         print(f"Subscribed to: {TOPIC}")
18     else:
19         print(f"Connection error: {rc}")
20
21 def on_message(client, userdata, msg):
22     try:
23         payload_str = msg.payload.decode()
24
25         # Sends message via UDP
26         sock.sendto(msg.payload, (IP_DESTINO, PORTA_DESTINO))
27
28     except Exception as e:
29         print(f"Error processing/sending message: {e}")
30
31 # Client Initialization
32 client = mqtt.Client()
33 client.on_connect = on_connect
```

```

34 client.on_message = on_message
35
36 print(f"Connecting to {BROKER}...")
37 client.connect(BROKER, 1883, 60)
38 client.loop_forever() # Keeps the script running

```

A.2 Subscriber Own Script (subscriberOwn.py)

```

1  import paho.mqtt.client as mqtt
2  import json
3  import datetime
4  import time
5  import socket
6
7  BROKER = "localhost"
8  TOPIC = "vanetza/own/cam"
9
10 IP_DESTINO = "192.168.6.10"
11 PORTA_DESTINO = 5000
12
13 sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
14
15 last_update_time = 0
16
17 def on_connect(client, userdata, flags, rc):
18     if rc == 0:
19         client.subscribe(TOPIC)
20         print(f"Subscribed to: {TOPIC}")
21     else:
22         print(f"Connection error: {rc}")
23
24 def on_message(client, userdata, msg):
25     global last_update_time
26     current_time = time.time()
27
28     # 1 second limit
29     if current_time - last_update_time < 1.0:
30         return
31
32     last_update_time = current_time
33
34     try:
35         payload_str = msg.payload.decode()
36         sock.sendto(msg.payload, (IP_DESTINO, PORTA_DESTINO))
37     try:

```

```

38         data = json.loads(payload_str)
39         formatted_data = json.dumps(data, indent=4, sort_keys=True)
40     except json.JSONDecodeError:
41         formatted_data = payload_str
42
43     print(f"--- Message sent: {IP_DESTINO}:{PORTA_DESTINO} (Tópico:
44           {msg.topic}) ---")
45     print(formatted_data)
46     print("-" * 30)
47
48     except Exception as e:
49         print(f"Error processing/sending message: {e}")
50
51 client = mqtt.Client()
52 client.on_connect = on_connect
53 client.on_message = on_message
54
55 print(f"Connecting to {BROKER}...")
56 client.connect(BROKER, 1883, 60)
57
58 client.loop_forever()

```

A.3 Mock Collision Test Script (mockCollisionTest.py)

```

1  import socket
2  import json
3  import time
4  import math
5  import copy
6
7  # UDP Destination
8  IP_DESTINO = "192.168.6.10"
9  PORTA_DESTINO = 5000
10 FREQUENCIA_ENVIO = 0.1
11 COMPENSACAO_ICONE = 0
12
13 A1_LAT = 38.736407 # Coordinates of the board 144
14 A1_LON = -9.300935
15 A2_LAT = 38.735913
16 A2_LON = -9.300948
17
18 B1_LAT = 38.735981 #Coordinates of the other car
19 B1_LON = -9.301366
20 B2_LAT = 38.735971

```

 Python

```

21 B2_LON = -9.300761
22
23 PACOTE_BASE = {
24     "driveDirection": "FORWARD", "timestamp": 0, "newInfo": True, "rssi":
25     -255,
26     "stationID": 0, "receiverID": 144, "receiverType": 5, "packet_size": 0,
27     "generationDeltaTime": 22742, "stationType": 5, "latitude": 0,
28     "longitude": 0,
29     "semiMajorConf": 4095, "semiMinorConf": 4095, "semiMajorOrient": 3601,
30     "altitude": 227.1, "altitudeConf": 11, "heading": 0.0, "headingConf":
31     127.0,
32     "speed": 10.0, "speedConf": 0.24, "length": 10.0, "width": 3.0,
33     "acceleration": 0.0,
34     "curvature": 1023, "yawRate": 0.0, "brakePedal": True, "gasPedal": False,
35     "emergencyBrake": True, "collisionWarning": True, "accEngaged": True,
36     "cruiseControl": True, "speedLimiter": True, "specialVehicle": None,
37     "test": {"json_timestamp": 0, "wave_timestamp": 0}
38 }
39
40 sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
41
42 def calcula_posicao(lat1, lon1, lat2, lon2, t):
43     progresso = (math.sin(t) + 1) / 2
44     lat = lat1 + (lat2 - lat1) * progresso
45     lon = lon1 + (lon2 - lon1) * progresso
46     return lat, lon
47
48 def get_bearing(lat1, lon1, lat2, lon2):
49     dLon = math.radians(lon2 - lon1)
50     l1 = math.radians(lat1)
51     l2 = math.radians(lat2)
52     y = math.sin(dLon) * math.cos(l2)
53     x = math.cos(l1) * math.sin(l2) - math.sin(l1) * math.cos(l2) *
54     math.cos(dLon)
55     bearing = math.degrees(math.atan2(y, x))
56     return (bearing + 360) % 360
57
58 ANG_A_IDA = get_bearing(A1_LAT, A1_LON, A2_LAT, A2_LON)
59 ANG_A_VOLTA = get_bearing(A2_LAT, A2_LON, A1_LAT, A1_LON)
60
61 ANG_B_IDA = get_bearing(B1_LAT, B1_LON, B2_LAT, B2_LON)
62 ANG_B_VOLTA = get_bearing(B2_LAT, B2_LON, B1_LAT, B1_LON)
63
64 def enviar_pacote(carro_nome, station_id, lat, lon, heading):
65     pacote = copy.deepcopy(PACOTE_BASE)
66     agora = time.time()
67     pacote["timestamp"] = agora

```

```

63     pacote["test"]["json_timestamp"] = agora
64     pacote["test"]["wave_timestamp"] = agora
65     pacote["latitude"] = lat
66     pacote["longitude"] = lon
67     pacote["stationID"] = station_id
68     pacote["heading"] = round(heading_corrigido, 2)
69     payload = json.dumps(pacote).encode('utf-8')
70     sock.sendto(payload, (IP_DESTINO, PORTA_DESTINO))
71     tempo_simulacao = -math.pi / 2
72     velocidade_simulacao = 0.5
73
74     try:
75         while True:
76             pos_A_lat, pos_A_lon = calcula_posicao(A1_LAT, A1_LON, A2_LAT,
77             A2_LON, tempo_simulacao)
78             pos_B_lat, pos_B_lon = calcula_posicao(B1_LAT, B1_LON, B2_LAT,
79             B2_LON, tempo_simulacao)
80
81             # The cosine dictates the direction (positive = forward, negative =
82             # return)
83             if math.cos(tempo_simulacao) >= 0:
84                 heading_A = ANG_A_IDA
85                 heading_B = ANG_B_IDA
86             else:
87                 heading_A = ANG_A_VOLTA
88                 heading_B = ANG_B_VOLTA
89
90             enviar_pacote("144", 144, pos_A_lat, pos_A_lon, heading_A)
91             enviar_pacote("145", 145, pos_B_lat, pos_B_lon, heading_B)
92
93             tempo_simulacao += velocidade_simulacao * FREQUENCIA_ENVIO
94             time.sleep(FREQUENCIA_ENVIO)
95
96     except KeyboardInterrupt:
97         print("\nSimulação parada.")
98     finally:
99         sock.close()

```

A.4 Collision detection

```

1     private fun checkCollisions(): Boolean {
2         if (locationLine == null || locationLine!!.latLongs.size < 2) return
3         false
4         val myStart = locationLine!!.latLongs[0]
5         val myEnd = locationLine!!.latLongs[1]

```

 Python

```

6      for ((id, data) in otherVehicles) {
7          val theirStart = data.center
8          val theirLine = data.line
9          val theirPolygonPoints = data.polygon.latLongs
10
11         // 1. A nossa linha cruza-se com o retângulo de bounding do
            vizinho?
12         if (theirPolygonPoints.size >= 4) {
13             // Testar intersecção com cada segmento do polígono
                (retângulo)
14             for (i in 0 until 4) {
15                 val p3 = theirPolygonPoints[i]
16                 val p4 = theirPolygonPoints[(i + 1) % 4]
17                 if (segmentsIntersect(myStart, myEnd, p3, p4)) {
18                     return true
19                 }
20             }
21         }
22
23         // 2. A nossa linha cruza-se com a linha de predição deles?
24         if (theirLine != null && theirLine.latLongs.size >= 2) {
25             val theirEnd = theirLine.latLongs[1]
26             if (segmentsIntersect(myStart, myEnd, theirStart, theirEnd))
                {
27                 return true
28             }
29         }
30     }
31     return false
32 }

```

Appendix B | AI Usage Declaration

It is hereby declared that all technical decisions, implementation logic, and core content are entirely of the authors' own creation and responsibility. Within the scope of this work, Artificial Intelligence tools were utilized to support the development of the report, specifically focusing on linguistic revision to ensure grammatical correctness, clarity, and a formal academic tone in English. Additionally, AI assisted in structuring the bibliography by generating BibTeX entries for software utilities, industry standards, and scientific papers, ensuring compliance with academic referencing requirements. Furthermore, these technologies were applied to generate specific illustrative images for the report and supported the application development process, including refining user interface design details and implementing collision and overtaking algorithms. Finally, AI tools helped construct test scripts, improve Python code efficiently, summarize complex technical content, and formulate concise descriptions for optimizing information synthesis while preserving the scientific rigor of the document